Progettazione e implementazione di un Hybrid Multilayer Firewall

Costanza Mancini (637274)

10 novembre 2026

Indice

1 Obiettivo del progetto **2**

2 Architettura e flusso di elaborazione **2**
2.1 Organizzazione dei file 2

3 Scelte progettuali **3**

4 Analisi del traffico **3**
4.1 Analisi di pacchetto 4
4.2 Analisi di flusso 4

5 Policy Engine **5**
5.1 Politica e configurazione 5
5.2 Regole per singolo pacchetto 6
5.2.1 Regole per flusso 6
5.2.2 Regole per finestre temporali 6
5.3 Esito della valutazione 7

6 Testing e risultati **7**
6.1 Ambiente sperimentale e profili 7
6.2 Scenari e risultati 7

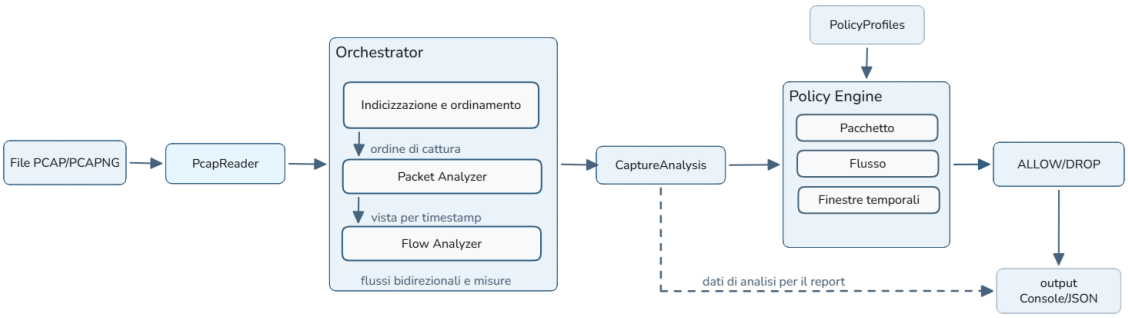
7 Conclusioni **8**

1 Obiettivo del progetto

Il progetto prevede la progettazione e l'implementazione di un *Hybrid Multilayer Firewall*, combinando l'analisi del traffico di rete con una serie di decisioni di filtraggio personalizzate. L'intenzione è quella di sfruttare la raccolta dei dati del traffico e la loro analisi a scopi di sicurezza, per individuare anomalie o attacchi specifici.
La decisione principale è stata affiancare due livelli di analisi, quella dei singoli pacchetti e quella di flusso. L'elaborazione viene svolta in locale e offline su traffico precedentemente acquisito (tramite strumenti come Wireshark) e salvato all'interno di file PCAP.
Il componente fondamentale del firewall è rappresentato dal *Policy Engine* che, a partire dalle informazioni raccolte, applica un insieme di regole stabilite per assegnare ad ogni pacchetto un esito di tipo ALLOW o DROP. Il comportamento del sistema è stato valutato principalmente tramite delle suite di scenari sintetici e mirati, di cui sono riportati i risultati, ma anche da una serie di catture di traffico di una rete domestica.

2 Architettura e flusso di elaborazione

L'architettura del firewall è stata progettata su più componenti, ognuno con una responsabilità ben precisa nel flusso di elaborazione delle informazioni catturate.
L'Orchestrator è l'elemento che si occupa del coordinamento delle operazioni, stabilendo il corretto ordine delle fasi e mantenendo il collegamento tra il pacchetto catturato, il suo eventuale flusso di appartenenza e la decisione finale.
I pacchetti acquisiti vengono dati in pasto ad una funzione che utilizza PcapReader che li elabora uno alla volta per prenderne i campi d'interesse, come indirizzi MAC e IP, protocollo di trasporto, porte, flag TCP, timestamp e lunghezze disponibili. Le informazioni estratte sono raccolte in un oggetto Packet, che mantiene l'indice utile a identificarne la posizione temporale nel PCAP. Il Packet Analyzer riceve tutti i pacchetti nell'ordine originale di cattura e produce per ciascuno un risultato dettagliato con i campi osservati e l'esito dei controlli effettuati.
L'orchestratore si occupa di costruire una vista dei pacchetti ordinata per timestamp. Il Flow Analyzer riceve la vista e per ogni pacchetto, quando possibile, individua il flusso di appartenenza. Si riesce a costruire un collegamento per risalire dal singolo pacchetto alle statistiche di flusso elaborate dall'analyzer, raccolti in un oggetto Flow.
All'interno di un CaptureAnalysis i pacchetti vengono raccolti, in base all'ordine di cattura, con i risultati dei controlli. Il Policy Engine applica la configurazione di filtraggio stabilita e produce per ciascun pacchetto un giudizio affermativo o negativo, insieme alla relativa motivazione.



2.1 Organizzazione dei file

Il codice del firewall è organizzato in moduli Python con responsabilità distinte.

- `main.py`: punto di ingresso del programma che gestisce i parametri da riga di comando e la selezione del profilo;
- `tools/pcap_reader.py`: legge e decodifica i file PCAP/PCAPNG con Scapy;
- `firewall/orchestrator.py`: coordina le fasi dell'elaborazione;

- `firewall/analysis/`: contiene i componenti per l'analisi del traffico
  - `packet_analyzer.py`: verifica caratteristiche, validità e coerenza dei singoli pacchetti;
  - `flow_analyzer.py`: contiene la fase di aggregazione dei pacchetti in flussi bidirezionali per poi calcolarne le statistiche;
  - `util.py`: contiene funzioni di utilità condivise tra gli analyzer;
- `firewall/engine/`: contiene logica e configurazione del filtraggio
  - `policy_engine.py`: implementa le regole su pacchetti, flussi e finestre temporali, producendo le decisioni ALLOW/DROP;
  - `policy_config.py`: definisce di parametri modificabili, autorizzazioni e soglie numeriche utilizzate dall'engine;
  - `policy_profile.py`: gestisce i profili di configurazione personalizzabili per il firewall;
  - `policy_decision.py`: definisce le strutture delle decisioni in output e le relative motivazioni;
- `firewall/schemas/`: contiene le strutture dati definite e utilizzate dai componenti
  - `packet.py`, `packet_analysis.py`: rappresentano pacchetti e risultati dell'analisi;
  - `flow.py`, `flow_analysis.py`: rappresentano flussi e risultati con statistiche e controlli;
  - `indexed_packet.py`, `flow_process_result.py`, `capture_analysis.py`: inseriti per mantenere le associazioni tra pacchetti, flussi e risultati della cattura
- `firewall/reporting/`: contiene i moduli per la presentazione dei risultati su console e l'esportazione dei report JSON;
- `data/`: contiene i test fatti sul sistema per verificarne il corretto funzionamento.

3 Scelte progettuali

Per l'implementazione del sistema, al fine di prediligere semplicità nella prototipazione, è stato scelto Python come linguaggio. Il suo utilizzo ha permesso di definire in modo chiaro le strutture dati scambiate tra i componenti e utilizzare di conseguenza la libreria Scapy per la lettura e la decodifica dei pacchetti. Scapy mette a disposizione un'interfaccia per accedere direttamente a diversi livelli protocollari e ai relativi campi attraverso poche istruzioni. Si utilizza la funzione PcapReader per la lettura progressiva dei PCAP, utile per contenere l'occupazione della memoria durante la lettura dei file, soprattutto nel caso di catture realistiche voluminose, senza richiederne il caricamento integrale.
Si è preferito sviluppare il progetto come un sistema di analisi offline e locale soprattutto allo scopo di contenere la complessità implementativa. L'elaborazione offline consente di calcolare caratteristiche aggregate ricavabili dai flussi, senza il vincolo dell'effettiva velocità di trasmissione dei pacchetti e di ripetere l'elaborazione della stessa cattura con configurazioni di filtraggio diverse.
L'utilizzo dei file PCAP per conservare tracciati di traffico reali o semplificati è stata una conseguenza di questi aspetti, in modo da memorizzare informazioni di rete di pacchetti precedentemente acquisiti, conservandone la struttura originale.

4 Analisi del traffico

L'analisi del traffico di rete è stata fatta su due livelli in modo da distinguere le informazioni ricavabili dall'elaborazione del singolo pacchetto da quelle che richiedono l'osservazione nel tempo di aggregazioni. I risultati ottenuti vengono mantenuti distinti grazie alle strutture dati dedicate e successivamente dati in input per le decisioni di filtraggio.

4.1 Analisi di pacchetto

L'analisi di pacchetto ha l'obiettivo di verificare la coerenza dei campi disponibili e di riportare caratteristiche osservabili. Il PacketAnalyzer riceve un oggetto Packet ed effettua una serie di controlli. Ogni analisi di pacchetto restituisce un oggetto PacketAnalysis contenente informazioni generali con indirizzi e protocolli, esiti dei controlli e caratteristiche significative rilevate. Gli esiti sono:

- VALID: i dati rispettano i vincoli;
- INVALID: i dati presentano una violazione sui controlli;
- INCOMPLETE: i dati disponibili non consentono di completare la verifica.

Casi in cui i controlli non sono pertinenti restituiscono esito NOT APPLICABLE mentre per controlli non eseguiti si dà esito NOT CHECKED.

Controllo	**Verifica effettuata**
Timestamp	Presenza e validità
Lunghezze del pacchetto	Coerenza tra byte acquisiti, dimensione originale e lunghezza dichiarata
Payload	Presenza e coerenza della dimensione dei dati
Coerenza protocollare	Corrispondenza delle informazioni tra diversi livelli protocollari
Indirizzi IP	Validità del formato e corrispondenza in base alla versione IPv4/IPv6
Intestazione IP	Verifica della struttura IPv4/IPv6
IPv6 Next Header	Coerenza della catena delle intestazioni di estensione
Frammentazione IP	Riconoscimento di frammenti IPv4/IPv6
Porte TCP/UDP	Validità e classificazione delle porte sorgente e destinazione
TCP	Completezza di campi e flag e lunghezza dell'intestazione
UDP	Controllo della lunghezza minima di 8 byte e coerenza della dimensione del datagramma.
ICMP/ICMPv6	Disponibilità di tipo e codice e lunghezza del messaggio

4.2 Analisi di flusso

L'analisi basata sui flussi di rete effettua verifiche sulle relazioni di pacchetti appartenenti alla stessa comunicazione, ricavandone misure statistiche quantitative e temporali.
I pacchetti vengono raggruppati in base alla FlowKey, una chiave univoca e indipendente dalla direzione del traffico, che identifica il flusso, composta da versione IP, numero di protocollo IP, protocollo di trasporto identificato e i due endpoint coinvolti.
La normalizzazione degli indirizzi e la disposizione degli endpoint in ordine canonico vengono fatte per rappresentare il flusso in modo bidirezionale, pur mantenendo contatori di pacchetti e byte separati per le due direzioni.
Per ogni flusso sono raccolte le seguenti misure:

Misura	**Costruzione**	**Unità**
Packet Count e Byte Count	N pacchetti e B byte complessivi	pkt, byte
Directional Counts	Gli stessi conteggi calcolati separatamente per direzione	pkt, byte
Min, Max e Mean Packet Size	Minimo e massimo di L_i e media B/N	byte
First Seen e Last Seen	Primo e ultimo timestamp del flusso (t_1 e t_N)	seconds
Flow Duration	$D = t_N - t_1$	seconds
Packet Rate medio	N/D per $D > 0$	pkt/s
Max Packet Rate	Massimo conteggio in una finestra mobile di 1 s	pkt/s
Byte Rate medio	B/D per $D > 0$	byte/s
Mean Inter-Packet Interval	Media degli intervalli $t_i - t_{i-1}$	seconds
Std Inter-Packet Interval	Deviazione standard degli intervalli $t_i - t_{i-1}$	seconds
TCP Flag Counts	Occorrenze dei flag SYN, FIN e RST nei flussi TCP	events

Il rate medio descrive l'intero flusso mentre il rate massimo descrive il tratto più intenso: ad ogni pacchetto di tempo t sono conservati i timestamp dell'istanza in $(t - 1\,\mathrm{s}, t]$, contando anche il pacchetto corrente e aggiornando il massimo osservato fino a quel momento. Il conteggio include entrambe le direzioni e considera singolarmente i pacchetti con timestamp uguali.
Tutte le statistiche derivate sono calcolate alla finalizzazione del flusso, in caso di inattività o al termine della cattura. Oltre alle misure statistiche, il Flow Analyzer mantiene informazioni relative alla finalizzazione del flusso e allo stato della comunicazione TCP.

Informazione	**Descrizione**
`finalized_at`	Timestamp a cui viene finalizzato il flusso
`finalization_reason`	Motivo della finalizzazione (inattività o termine della cattura)
`handshake_observed`	Controllo su handshake completato
`tcp_state`	Stato osservato della comunicazione TCP durante handshake
`tcp_state_anomalies`	Numero di anomalie rilevate nella sequenza di apertura della connessione TCP

Considerando $t_1, \ldots, t_N$ i timestamp dei pacchetti nel flusso, ordinati cronologicamente, e L_i la lunghezza originale del pacchetto i riportata dalla cattura (packet_length):

- il volume $B = \sum_{i=1}^{N} L_i$ usa questa lunghezza anche quando il numero di byte acquisiti (captured_length) è inferiore;
- la durata osservata $D = t_N - t_1$ si misura tra i pacchetti del flusso;
- il timestamp di finalizzazione può essere successivo a t_N: un flusso viene chiuso per inattività (`TIMEOUT`, soglia predefinita di 180s) o al termine della cattura (`END_OF_CAPTURE`); a seguito del timeout, la stessa chiave può identificare una nuova istanza di flusso.

Per quanto riguarda il controllo sugli handshake, la verifica si basa su un SYN iniziale che identifica chi avvia la comunicazione, un SYN+ACK nella direzione opposta che registra la risposta e un ACK finale che completa l'apertura. Il risultato riporta lo stato (`tcp_state`), l'esito dell'handshake (`tcp_handshake_status`), l'eventuale reset e il numero di anomalie. Per la prima anomalia registra anche tipo, stato precedente, direzioni attesa e osservata, flag e posizione nel flusso. La ricostruzione si limita all'apertura osservata e al reset, senza validare i numeri di sequenza o seguire la chiusura completa della connessione.

5 Policy Engine

Il Policy Engine rappresenta il nucleo decisionale del firewall. Riceve gli esiti delle analisi e li confronta con un insieme di regole e soglie definite nella configurazione, stabilendo un esito positivo o negativo sull'applicazione della policy.

5.1 Politica e configurazione

Si applica una politica di default-deny: ogni pacchetto viene quindi autorizzato solo se soddisfa un servizio esplicitamente ammesso. La policy stabilisce delle autorizzazioni di servizio e un insieme di limiti quantitativi e condizioni di blocco:

- una regola di autorizzazione specifica protocollo, indirizzi, reti di origine e destinazione, eventuali porte TCP/UDP oppure tipi e codice ICMP;
- i limiti definiti stabiliscono una serie di valori massimi per le misure aggregate dei flussi;
- le condizioni di blocco invece specificano quando va prodotto un esito DROP: se più controlli bloccanti scattano sullo stesso pacchetto, il primo nell'ordine di valutazione determina la regola principale e gli altri sono riportati come condizioni secondarie.

5.2 Regole per singolo pacchetto

Il primo livello di regole considera i campi osservati e gli esiti delle verifiche strutturali.

Regola	**Condizione di blocco**
Timestamp	Timestamp assente, non numerico, negativo o non finito
Restrizioni IP	Indirizzi o reti vietati, sorgenti multicast/broadcast, loopback non coerente e sorgenti esterne ai prefissi WAN ammessi
Restrizioni porte	Porte TCP/UDP vietate nella configurazione, per sorgente o destinazione
Controlli strutturali	Campi invalidi o requisiti di validità non soddisfatti
`UNSUPPORTED_TUNNEL`	Incapsulamenti IPv4-in-IP, IPv6-in-IP o GRE non supportati
`IP_FRAGMENT`	Frammentazione IP non gestita
`TCP_SYN_FIN`	Flag SYN e FIN contemporaneamente attivi
`TCP_SYN_RST`	Flag SYN e RST contemporaneamente attivi
`TCP_NULL_FLAGS`	Nessun flag TCP attivo
`L3_SIZE_LIMIT`	Dimensione IP superiore al massimo configurato in byte

5.2.1 Regole per flusso

Le regole riguardanti il flusso utilizzano le statistiche dell'istanza bidirezionale finalizzata per produrre informazioni generali sui pacchetti che vi appartengono. Le soglie in pacchetti e byte al secondo sono applicate quando il flusso raggiunge una durata minima e un numero minimo di pacchetti.
L'engine riporta anche delle segnalazioni come evidenze diagnostiche, riguardanti l'handshake TCP, senza determinare il blocco della comunicazione.

Regola	**Condizione di blocco**	**Unità**	**Esito**
`FLOW_PACKET_RATE`	Packet rate medio superiore al limite	pkt/s	DROP
`FLOW_PEAK_PACKET_RATE`	Picco superiore al limite	pkt/s	DROP
`FLOW_BYTE_RATE`	Byte rate medio superiore al limite	byte/s	DROP
`FLOW_DURATION`	Durata del flusso superiore al limite	second	DROP
`FLOW_BYTE_COUNT`	Volume complessivo superiore al limite	byte	DROP
`TCP_HANDSHAKE_INCOMPLETE`	Apertura osservata e rimasta in attesa del SYN+ACK o dell'ACK conclusivo	-	alert
`TCP_STATE_ANOMALY`	Incoerenza accertata nella sequenza di apertura TCP	-	alert

5.2.2 Regole per finestre temporali

Il terzo livello di regole valuta gli eventi osservati fino ad un certo pacchetto, a prescindere dall'appartenenza ai flussi. I contatori vengono organizzati in base agli endpoint e al protocollo coinvolto, secondo la regola applicata.
Lo scopo è individuare concentrazioni di eventi in un determinato intervallo di durata configurabile. I pacchetti sono elaborati in ordine di timestamp. Ad ogni evento di tempo t scadono quelli a tempo minore o uguale a $t - W$, con W la durata configurata. L'engine inserisce l'evento corrente e valuta la finestra $(t - W, t]$.

Regola	**Valori misurati**	**Unità**
`WINDOW_PACKET_FLOOD`	Numero di pacchetti associabili alla chiave	pkt
`WINDOW_SYN_FLOOD`	Numero di SYN senza ACK con payload = 0	events
`WINDOW_ECHO_FLOOD`	Numero di richieste ICMP Echo valide	events
`WINDOW_PORT_SCAN`	Numero di porte destinazione TCP distinte	ports
`WINDOW_UDP_PORT_SCAN`	Numero di porte destinazione UDP distinte	ports
`WINDOW_HOST_SCAN`	Numero di IP destinazione distinti	hosts
`WINDOW_SMALL_PACKETS`	Numero di pacchetti con dimensione IP entro i limiti	pkt

Per ogni porta o host distinto sono mantenute il numero di occorrenze nella finestra: le soglie di flooding e piccoli pacchetti producono un blocco quando il numero di eventi supera il massimo configurato; le scansioni si attivano al raggiungimento del numero di destinazioni distinte stabilito.

5.3 Esito della valutazione

Per ogni pacchetto, l'engine restituisce una decisione tramite le etichette ALLOW e DROP, riportando identificativo della regola, motivazione, riscontri secondari e identificativi delle regole pertinenti non valutabili per insufficienza di informazioni.

6 Testing e risultati

6.1 Ambiente sperimentale e profili

Il funzionamento del firewall è stato verificato tramite catture PCAP sintetiche generate con Scapy. Ogni cattura viene elaborata attraverso l'intera pipeline, dalla lettura dei pacchetti alla produzione delle decisioni. I risultati sono stati riportati all'interno di riepiloghi in JSON, contenenti gli ALLOW/DROP, le regole e ulteriori riscontri associati, per ogni pacchetto.
La configurazione è fatta tramite profili selezionabili all'avvio del programma, che permettono di variare le autorizzazioni di servizio, le restrizioni su indirizzi e porte e i limiti sulle soglie, senza modificare il codice del motore. Sono previsti un profilo dimostrativo (`demo`), uno personalizzabile (`custom`) e uno dedicato alle verifiche (`test`).
Si possono ripetere le analisi con il comando seguente, sostituendo il percorso della cattura e il profilo della suite:

```
python src/main.py cattura.pcap --profile PROFILO --json report.json
```

La prima serie di test con le 17 catture presenti `data/policy_test_pcaps/` utilizza il profilo `test1`. La seconda serie di test è invece composta da 21 PCAP elencati nel `manifest.csv` di `data/final_checks_tcp_validazione/` e fatta tramite il profilo `test2`: riguarda packet rate e byte rate, durata e timeout dei flussi, gestione IPv6, intestazioni di estensione, frammentazione, tunnel IP, catture troncate e riconoscimento dell'handshake TCP.

Parametro	`test1`	`test2`
Finestra della policy	2 s	-
Flood di pacchetti, SYN, Echo	30, 20, 20 eventi	-
Piccoli pacchetti	20 eventi, fino a 80 byte IP	-
Scansioni TCP/UDP e host	10 porte o 10 host distinti	-
Minimi per i rate medi	1 s e 20 pacchetti	1 s e 20 pacchetti
Packet rate medio massimo	10 000 pkt/s	150 pkt/s
Max Packet Rate	Disattivato	150 pkt/s su 1 s
Byte rate medio massimo	10 000 000 byte/s	20 000 byte/s
Durata massima del flusso	300 s	300 s
Volume massimo del flusso	60 000 byte	2 000 000 byte

6.2 Scenari e risultati

Sono riportati i risultati osservati nei 17 scenari iniziali indicando le conseguenti decisioni prodotte, con il profilo `test1`.

Test	**Scenario**	**Pkt**	**ALLOW**	**DROP**
01	Handshake e dati TCP	6	6	0
02	Richiesta e risposta UDP	2	2	0
03	Servizio non autorizzato	1	0	1
04	Flag TCP SYN+FIN	1	0	1
05	Flag TCP SYN+RST	1	0	1
06	TCP senza flag	1	0	1
07	SYN seguito da RST	2	2	0
08	Scansione porte TCP	12	9	3
09	Scansione porte UDP	12	9	3
10	Scansione host	12	9	3
11	SYN flood	40	20	20
12	ICMP Echo flood	40	20	20
13	Piccoli pacchetti UDP	40	20	20
14	Packet flood UDP	40	30	10

Test	Scenario	Pkt	ALLOW	DROP
15	ACK TCP ripetuti	40	30	10
16	SYN ripetuti stessa porta	12	12	0
17	Volume complessivo UDP	60	0	60

Si ha quindi un totale di 322 pacchetti testati, con 169 ALLOW e 153 DROP.
I primi test 01-02-03 verificano le autorizzazioni di servizi, la gestione delle risposte e l'applicazione della politica di default deny. I test 04-05-06 sui flag permettono di verificare che un servizio ammesso rimane comunque soggetto di vincoli di blocco; i test 08-09-10 sulle scansioni confermano il corretto funzionamento distinguendo eventi appartenenti a flussi diversi, contando le destinazioni distinte. Nei test 11-12-13 si osserva che il superamento della soglia produce 10 DROP mentre i successivi 10 sono attribuibili a situazioni di flooding. Nel caso 16, 12 SYN sulla stessa porta valgono una sola porta distinta. Nel test 17, il blocco di tutti i pacchetti dipende dalla valutazione del flusso finalizzato: quando si riscontra il superamento del volume massimo, si applica il giudizio DROP all'intera istanza del flusso.

I 21 test con il profilo `test2` hanno dato i seguenti risultati:

ID	Scenario	Pkt	ALLOW	DROP	Regola principale o riscontro
01	UDP a basso rate	15	15	0	Media 1,07 e picco 1 pkt/s. Il flusso ha meno di 20 pacchetti
02	UDP a packet rate elevato	900	0	900	`FLOW_PACKET_RATE`. Media 300,33 e picco 300 pkt/s
03	UDP a byte rate elevato	60	0	60	`FLOW_BYTE_RATE`. Circa 27 295 byte/s
04	Burst UDP su più flussi	300	300	0	300 flussi da un pacchetto, picco 1 pkt/s per flusso
05	TCP di lunga durata	305	0	305	`FLOW_DURATION`. 302 s. Handshake completo
06	TCP di breve durata	10	10	0	Durata 7 s e handshake completo.
07	Inattività tra aperture TCP	6	6	0	Due istanze di circa 0,02 s. La prima chiude per `TIMEOUT`
08	IPv6 TCP e UDP	4	4	0	Entrambi i servizi autorizzati. Handshake TCP completo
09	IPv6 Hop-by-Hop e UDP	1	1	0	Catena Next Header coerente e servizio UDP autorizzato
10	Frammenti IPv4 validi	4	0	4	`IP_FRAGMENT`
11	Frammento IPv4 incoerente	1	0	1	`INVALID` per `ip_fragmentation_consistency`. Blocco strutturale
12	Frammenti IPv6	2	0	2	`IP_FRAGMENT`. Il frammento successivo resta fuori dal flusso
13	Tunnel IPv4-in-IPv4	1	0	1	`UNSUPPORTED_TUNNEL`
14	Tunnel IPv6-in-IPv4	1	0	1	`UNSUPPORTED_TUNNEL`
15	Intestazione troncata	1	0	1	`DEFAULT_DENY`. Dati insufficienti per autorizzare il servizio
16	Payload troncato	1	1	0	`TEST_UDP_IPV4`. Controlli di lunghezza `INCOMPLETE`
17	Handshake regolare e dati	4	4	0	Handshake `COMPLETE`
18	SYN+ACK assente	3	3	0	`TCP_HANDSHAKE_INCOMPLETE`
19	SYN+ACK dall'iniziatore	3	3	0	`TCP_STATE_ANOMALY` e `TCP_HANDSHAKE_INCOMPLETE`
20	Due SYN, senza risposta	2	2	0	`TCP_HANDSHAKE_INCOMPLETE`. Ritrasmissione tollerata
21	TCP ad alto rate dopo handshake	903	0	903	`FLOW_PACKET_RATE`. Media 225,94 e picco 300 pkt/s

Su un totale di 2.527 pacchetti testati sono stati ottenuti 349 ALLOW e 2178 DROP.

7 Conclusioni

Il prototipo di firewall realizzato ha un comportamento coerente e corretto rispetto a tutte le regole stabilite e i limiti presenti. Ogni decisione dipende dai servizi autorizzati, dalle misure di flusso e dagli eventi rilevati nelle finestre temporali.

È interessante considerare una serie di sviluppi futuri possibili.
Il funzionamento locale e offline permette di ripetere l'analisi sulla stessa cattura e confrontare gli esiti delle regole. Questo aspetto sicuramente costituisce un limite nel reale funzionamento del sistema e si potrebbe considerare di implementare l'elaborazione in tempo reale e l'acquisizione di traffico da differenti punti remoti.
Un altro aspetto emerso è quello della scalabilità, legato al consumo di memoria: il lettore decodifica un pacchetto alla volte ma il sistema ne conserva le rappresentazioni in memoria. Il consumo di memoria quindi cresce proporzionalmente alla dimensione della cattura e con lo stato mantenuto per flussi e finestre temporali.
Nonostante tecnologie come gestione dell'incapsulazione o della frammentazione non siano state implementate per semplificazione, una versione futura potrebbe prevedere lo sviluppo di questi aspetti.
Un'introduzione particolarmente rilevante sarebbe quella di un terzo livello di analisi, basato sui protocolli applicativi. L'integrazione tramite il toolkit di nDPI permetterebbe di classificare i flussi, utilizzabile dal Policy Engine come un ulteriore criterio di definizione ancora più specifico.